Unix for Software Developers

Unix Overview

Benedict Reuschling

October 13, 2025



Unix Overview

The objective of this lecture is to familiarize oneself with the Unix operating system from the perspective of system developers. Our aim is to illustrate how Unix serves as an interface to hardware for the development of applications and system programs. To achieve this, we will initially highlight the most crucial aspects of Unix:

- the **shell**,
- the **editors** and
- the **file system**.

Subsequently, we will explore the **Shell** and delve into programming with the Unix toolset. Shell programs frequently serve as the interface to commercial applications during the development of application software.

Overview

1 Introduction
- Unix History
- The Unix Philosophy

2 Overview
- Logging into the Unix System
- The Shell
- Files in Unix
- Getting help from the System

Introduction

Software can be categorized into two main areas:

System software This directly controls the computer.

Applications Designed to solve specific tasks given by the user.

The pivotal element of system software is the **operating system**, which governs the computer's resources and serves as the foundation for application development. A contemporary computer comprises the following components:

- one or more processors,
- main memory,
- cycle generators,
- disk drives,
- network components and
- peripheral components (keyboard, mouse, other input devices).

All together, they form a complex piece of a system.

Separation into layers

The objective of systems programming is to shield users from the intricacies of hardware complexity. To manage this complexity, the computer is conceptually divided into different **layers**.
At the core is the hardware layer, encompassing the physical devices with integrated components, wires, transistors, and more. Above it is a simple layer of software, known as firmware, directly interfacing with these physical devices. This firmware is embedded firmly into ROM (read-only memory) and serves as an interpreter for elementary machine instructions such as ADD, MOVE, JUMP collectively forming the machine language or Assembler.
Beyond the operating system layer, there exist tools and utilities essential for developing application software, such as C compilers, Java development environments, vi, or the bash shell. This layering model illustrates that an operating system acts as the bridge between hardware and the development of application systems.

Overview

1 Introduction
Unix History
The Unix Philosophy

2 Overview
Logging into the Unix System
The Shell
Files in Unix
Getting help from the System

Unix History

Unix originated in **1969** at **Bell Labs**, not as a product for commercial sale, but as a utility application to assist in programming.
The catalyst for Unix was the "space travel" project, which simulated the movement of celestial bodies in the solar system. Initially developed on a GE645 under the **MULTICS** operating system, the working program was transferred via punch cards to a PDP7, where it was put into production. However, the target system lacked a program development environment. Consequently, **Ken Thompson** created UNIX to enable direct program development on the PDP7.
The inaugural version of the UNIX operating system was uncomplicateda single-user system with a rudimentary user interface. The name "UNIX" is a play on the term for the multi-user system MULTICS, signifying **unique MULTICS**.

Unix History continued

A small group undertook the development of UNIX, a departure from the norm for operating systems at the time. Their **objective** was to craft a system that provided an optimal environment for programming. Until then, most operating systems were tailored to the specific hardware of a given system, emphasizing efficient resource utilization.
This unconventional approach led to the creation of an exceptionally advanced system, a fact underscored by the adoption of certain UNIX components by other standard systems of that era, such as MS-DOS.
Despite its advancements, UNIX might not have achieved widespread adoption if not for the evolution of its implementation language. The initial UNIX version was entirely written in the machine language of the PDP7. While such a system is fast, it lacks portability. This limitation changed with the development of the programming language called B.

History of Unix: 1970 – 1980s

In 1970, UNIX underwent a rewrite in the programming language **B**, spearheaded by **Dennis Ritchie**, rendering it portable to other systems. Ritchie subsequently refined B, elevating the programming language **C** to a higher level that nearly matched the efficiency of assembly language. The testament to the operating system's portability emerged in 1977 when UNIX made its appearance on the Interdata 8/32.
For their contributions, Thompson and Ritchie were honored with the ACM A. M. Turing Award in 1983, often referred to as the "Nobel Prize" of computer science, recognizing *"the clever selection of a few key thoughts and their elegant implementation in the development of UNIX and C"*. U.S. President Bill Clinton awarded the National Medal of Technology to Ritchie and Thompson for their pivotal role in the creation of Unix and C.

History of Unix: 1970 – 1980s

The parent company of Bell Labs, AT&T, where Thompson and Ritchie were based, faced restrictions from entering new markets, including the burgeoning computer market, due to U.S. consent decree acts. As a workaround, AT&T opted to sell copies of UNIX with the complete source code to universities at the cost of the data storage media of the time, which were tapes.

Universities embraced this offering enthusiastically for research purposes. The availability of the source code enabled further enhancements to the system. Particularly, at the University of California in Berkeley, where Ken Thompson served as a guest professor from 1976, significant improvements were made. Under the leadership of Bill Joy (later a co-founder of Sun Microsystems), Berkeley released its own Unix distribution in 1977, named **B**erkeley **S**oftware **D**istribution (BSD).

> *"There are two major products that come out of Berkeley: LSD and UNIX. We don't believe this to be a coincidence."*
> *- Jeremy S. Anderson*

History of Unix: 1970 – 1980s

Many Unix versions were initially developed on the PDP-11, which had a limited address space. The introduction of 32-bit address spaces on machines like the VAX-11/780 made it feasible to incorporate virtual memory and network functionalities. Consequently, Unix was ported to these machines, underscoring the enduring importance of **portability** in the system's design.

The research group at Bell Labs successively released the Unix Time Sharing System V8, V9, and V10. Subsequently, the Unix Support Group (USG) assumed the development and released System III, followed by System V in 1983. Concurrently, a court order permitted AT&T to sell System V as well.

History of Unix: late 80s till today

The Unix Support Group (USG) underwent several name changes until its acquisition by Novell in 1993. Novell later sold the UNIX trademark to the X/OPEN Consortium, granting them the authority to establish certification standards for Unix-based products. Subsequently, Novell sold Unix to the Santa Cruz Operation (SCO).
During the 1980s, Unix experienced significant commercialization, accompanied by increasingly restrictive licensing from AT&T. Simultaneously, the University of Berkeley continued releasing further versions of BSD, incorporating implementations of TCP/IP and reliable signals. These advancements drew the attention and funding of DARPA[1].
To mitigate the fragmentation of Unix's descendants, the POSIX[2] standardization project was founded. In 1988, they released their first standard, POSIX.1.

[1] Defense Advanced Research Projects Agency, a research agency of the US department of defense

[2] Portable Operating System Interface

Begin of the Open Source Movement

Richard Stallman's dissatisfaction with the increasing commercialization of Unix, where the source code was no longer available, prompted him to initiate the development of his own Unix-like operating system, known as GNU (GNU's Not Unix), starting in 1983. The concept of free software gained momentum from 1985 onward, and the GNU General Public License (GPL) was introduced to ensure that source code released under it remained open and freely accessible.
The GNU project successfully redeveloped many Unix software components. However, a crucial element was missing to complete a Unix-like system: the kernel. The GNU project attempted to implement a microkernel called HURD, but it proved more challenging in terms of development and debugging than initially anticipated.

The Dawn of GNU Linux

In 1987, Professor **Andrew S. Tanenbaum** developed **Minix**, a Unix-like operating system designed for teaching purposes. The motivation behind Minix's creation was partly the restrictive AT&T license, which hindered Tanenbaum's students from gaining a deeper understanding of operating system fundamentals. Although Minix did not gain significant prominence as a system on its own, it served as a source of inspiration for **Linus Torvalds**.

Torvalds, while studying computer science at the University of Helsinki, developed a kernel named **Linux** in 1991. Released under the GPL and distributed freely on the Internet, Linux provided the missing piece that the GNU project required. The combination of Linux and the GNU system enabled the installation and operation of a complete Unix system, commonly known as GNU/Linux.

The open nature of GNU components allowed individuals to combine them as they saw fit, leading to the proliferation of various Linux distributions over time, each with distinct development goals and purposes.

The Change from BSD to Open Source

The popularity of the BSD Distribution became so significant that companies began developing their own products based on BSD's TCP/IP code. However, a hurdle existed: a license from AT&T was required for such endeavors, and it came with a hefty price tag. In response to this, there was a demand to separate the networking code, leading to the introduction of a "Networking Release" in 1989 with a more lenient license.

Although individuals still had to pay a $1000 fee to Berkeley for the tapes, a workaround emerged. Those who had already paid for the software could share it with others (e.g., via FTP) without additional charges. Despite this arrangement, hundreds of organizations chose to purchase the releases, providing financial support for further development.

> *"Berkeley had what we called "copycenter", which is "take it down to the copy center and make as many copies as you want"."*
> *- Dr. Marshall Kirk McKusick*

The Change from BSD to Open Source

Additional components of the BSD system were released under a license independent of AT&T, based on the functionality descriptions of these tools. Over a span of 18 months, most of the system tools underwent rewriting. While the kernel posed a more significant challenge, eventually, every line of code was rewritten, except for six files still governed by the old license. Once these remaining files were reworked, university lawyers approved a new, open license.

In January 1992, the **386/BSD** was introduced, widely disseminated as a straightforward FTP download. This release prompted a flood of feature requests and bugfixes, overwhelming the university's ability to keep up with support.

This release became the foundation for the **NetBSD** project in 1993, followed by the **FreeBSD** project a few months later. **OpenBSD** emerged as a distinct project in 1995, splitting from NetBSD.

By the year 2000, **Darwin** appeared, incorporating FreeBSD as the core operating system. It forms the basis of Apple Inc.'s present-day **macOS**, utilizing Mach as the kernel.

Overview

1 Introduction
- Unix History
- The Unix Philosophy

2 Overview
- Logging into the Unix System
- The Shell
- Files in Unix
- Getting help from the System

The Unix Philosophy

Basic ideas for the development of Unix and its programs played an important role in the success of Unix. Douglas McIlroy, inventor of the Unix pipe, formulated these ideas as follows:

> *This is the Unix philosophy:*
>
> - *Write programs that do one thing and do it well.*
> - *Write programs to work together.*
> - *Write programs to handle text streams, because that is a universal interface.*

During this lecture, we will discover these ideas and their implementation. As software developers in a Unix environment you should follow them in your own programs.

Overview

1 Introduction
Unix History
The Unix Philosophy

2 Overview
Logging into the Unix System
The Shell
Files in Unix
Getting help from the System

Unix Overview

UNIX operates as a multi-user operating system primarily designed for **program development** and **interactive use**. By default, it is **not suitable for running real-time applications** because process interrupts cannot guarantee fixed completion times.
The architecture of UNIX is organized into **layered components**. At the lowest layer is the **kernel**, housing the control software. The kernel manages I/O operations for programs, oversees main memory and processes, safeguards data, logs critical system events, and performs various other functions. Unlike a microkernel, the UNIX kernel is a monolithic construct, featuring C source code with relatively fewer lines of assembler code in comparison.

Unix Overview

Above the kernel lies a layer of **user processes** that engage with the user and delegate tasks to the kernel for supervision. These user processes communicate with the kernel by invoking "system calls." The kernel provides a variety of system calls, which are essentially consistent across all Unix systems and remain independent of peripheral devices. The POSIX[3] standard serves to standardize these system calls.
Within user processes, you find not only user applications but also the UNIX command line interpreter (**shell**) and daemons (services). The shell acts as a shell around the kernel, serving as a means for users to access system resources. It functions like any other program. There are specialized shells available for many applications, such as menu shells that facilitate system use for non-programmers.

[3] Portable operating system interface UNIX

Unix Overview

The shell is a versatile yet straightforward utility for programmers. Through a concise set of commands, along with the use of wildcards, string variables, procedures, and high-level language constructs like conditions (if-then-else, case) and loops (while, for), users can craft intricate sequences without the need for writing separate programs. This capability allows the creation of specialized instructions tailored to specific applications.
Commonly used **command sequences** stored in files can be executed by a single command, known as command procedures. Furthermore, commands and sequences have the flexibility to run in the background, enhancing the efficiency and versatility of the shell.

Unix Overview

In UNIX, one can manipulate the **input and output (I/O)** of programs through redirection. This allows the output of a program to be directed into a file, displayed on the screen, or sent to a printer without the program needing specific knowledge of these actions. Similarly, input can originate from the screen or a file. Programs can concurrently run, with the output of one program serving as the input for another, facilitated by the kernel's pipe mechanism, ensuring synchronization.
Another notable feature of UNIX is its hierarchical file concept, incorporating differentiated file protection for the user, a group, and all other system users. Devices, which function similarly to files, can be mounted and unmounted. Files and processes serve as the primary objects within the UNIX operating system. I/O operations for files and devices, as well as communication between processes (via pipes), are designed to be simple, efficient, and consistent.

Overview

1. Introduction
   - Unix History
   - The Unix Philosophy
2. Overview
   - Logging into the Unix System
   - The Shell
   - Files in Unix
   - Getting help from the System

Logging into the Unix System

Unix is a multiuser- and multitasking system. This means that users can work on one Unix machine simultaneously, as well as each user is able to run programs which execute simultaneously.

Logging into the Unix System

Before starting to work with the Unix system, users must log onto the system to protect files from access by other users. Logging on means entering a special username and password. This way, the system knows which user wants to work with the system and can give the user their personal work environment, including all private data.

```
login: <loginname>
password:
$
```

The system recognizes a successful login by the appearance of the shell prompt:
$
The configured prompt shows information like the name of the system, the name of the user, the current directory, or the date. The prompt shown above describes commands executed by normal users, while `root` is using the # prompt.
Do not enter these prompts as part of a command.

Overview

1 Introduction
- Unix History
- The Unix Philosophy

2 Overview
- Logging into the Unix System
- The Shell
- Files in Unix
- Getting help from the System

The Shell

When a command is entered after the Unix prompt, the operating system doesn't directly respond; instead, the command interpreter, known as the shell, takes charge. The shell serves as the interface to the operating system, encapsulating the Unix core and facilitating a more accessible interaction with it. The shell, being a regular program, allows users to switch between different command interpreters interchangeably.

The Shell

Common Shells are

- sh, the Bourne shell (available on every Unix system, forming the base shell),
- csh, the C shell,
- ksh, the korn shell,
- bash, the Bourne again shell (standard on Linux distributions),
- zsh, the Z shell.

Not every shell (except sh) is available on every system, but the system administrator can install it.

The Shell

We are going to use the so called Z shell. All programs shown will also work within bash. A new shell (csh here) starts via:

```
$ csh
%
```

and exited via:

```
% exit
$
```

or Ctrl + D:

```
% ^D
$
```

Set up the commandline for the shell like this:

```
command parameter1 parameter2 ...
```

Shell commands

Commands are executable programs. The parameters control the execution of these commands. Parameters prefix by a minus sign, sometimes two (in Linux). A command to output the date and current time is `date`.

```
$ date
Fri Aug  3 12:05:33 CEST 2012
$
```

A command to count words, lines and characters in a file is `wc` (word count).

```
$ wc /etc/passwd
    46      104     2430 /etc/passwd
$
$ wc -l /etc/passwd
    460 /etc/passwd
$
```

Overview

1 Introduction
- Unix History
- The Unix Philosophy

2 Overview
- Logging into the Unix System
- The Shell
- Files in Unix
- Getting help from the System

Files in Unix

Files hold information. The simplest way to create a file is the use of programs that create output on the screen (standard out) and redirecting it. The echo command reflects all it's arguments to standard out:

```
$ echo this is the text written to the screen
this is the text written to the screen
$
```

Write the output to a file using redirection (using the [>] key).

```
$ echo this is the text written to the file dat1 > dat1
$
```

Files in Unix

Display the contents of a file using cat (con**cat**enate):

```
$ cat dat1
this is the text written to the file dat1
$
```

Using cat as a simple **editor** when redirecting standard output and omitting the filename. Use Ctrl + D to exit.

```
$ cat > dat2
This is a text spanning three lines
2. line
last line
$ cat dat2
This is a text spanning three lines
2. line
last line
$
```

Adding text to a file or replacing it

Be careful:

A simple > overwrites the contents of a file!

```
$ cat dat3
Beginning
$ echo End > dat3
$ cat dat3
End
$
```

To add text to the end of a file (preserving the current content in the file) use two >:

```
$ cat dat4
Beginning
$ echo End >> dat4
$ cat dat4
Beginning
End
$
```

Files and Directories

Directories hold files. A directory itself is a file. This way, a hierarchical file system is possible. The command to list the contents of a file is `ls` (**list**).

```
$ ls -l
-rw-rw-r--   1 bcr        users                53 Dec 24 11:28 dat2
$
```

`ls` with the option `l` (long) lists the directory contents in a long format with the following information:

- Permissions
- Number of references to the file
- Name of the file's owner
- Group
- Size in Bytes
- Date and time of the last change
- Name of the file

Manipulating Files and Directories - Copy

The Unix cp command (copy) serves multiple purposes and is similar to copy commands on other systems. It duplicates a given file under a new name in the current directory or with the same name at a different directory.
The basic syntax is:

```
cp source target
```

Given a directory `myfiles` with a file named `foo` inside:

```
$ pwd
/home/bcr/myfiles
$ ls
foo
```

To create a copy of the file `foo` named `bar`, enter: `cp foo bar`
Confirm that the file exists as an exact copy of foo with `ls -l`:

```
$ ls -l
total 16
-rw-r--r--  1 bcr  staff  4 13 Okt 12:01 bar
-rw-r--r--  1 bcr  staff  4 13 Okt 11:49 foo
```

Manipulating Files and Directories - Copy

Copying a file to itself is not allowed:

```
$ cp foo foo
cp: foo and foo are identical (not copied).
```

When the target is an existing directory that does not yet contain a file with the same name, cp will create the file in that directory:

```
$ mkdir subdir
$ ls subdir
$ cp foo subdir
$ ls subdir
foo
```

Manipulating Files and Directories - Recursive Copies

Directories are also copied using cp. The difference is that directories and the files within them require the -R (recursive) parameter. Otherwise, the operation will fail with an error. To see what will be copied, add the optional -v (verbose) parameter as well.

Copying a directory without -R

```
$ cp -v subdir newdir
cp: subdir is a directory (not copied).
```

Copying a directory with -R

```
$ cp -vR subdir newdir
subdir -> newdir
subdir/foo -> newdir/foo
```

No matter how deeply nested any subdirectories might be, using the recursive option will make cp descent into each one and copy it.

Manipulating files and directories: Move

Moving a file to a different location within the filesystem is done using `mv`. It also serves as a way to rename a file without having to delete and then copy it first.
The syntax is as follows:

```
mv source target
```

Given our directory `myfiles` with `foo` inside:

```
$ pwd
/home/bcr/myfiles
$ ls
foo
```

To rename the file `foo` to baz, use this: `mv foo baz`
Again, confirm that the file has a new name now using `ls -l`:

```
$ ls -l
total 8
-rw-r--r--  1 bcr  staff  4 13 Okt 12:05 baz
```

Directories are renamed in the same way.

Manipulating files and directories: Move

Moving a file to a different filesystem location is similar in syntax to the cp command, except that it will be removed from its original location to the new one.

Our current directory contains the baz file as well as the subdir and newdir directories.

```
$ ls -l
total 8
-rw-r--r--  1 bcr  staff   4 13 Okt 12:01 baz
drwxr-xr-x  3 bcr  staff  96 13 Okt 13:50 newdir
drwxr-xr-x  3 bcr  staff  96 13 Okt 12:37 subdir
```

The verbose parameter -v shows the result of moving baz to newdir:

```
$ mv -v baz newdir
baz -> newdir/baz
$ ls -l
total 0
drwxr-xr-x  4 bcr  staff  128 13 Okt 14:33 newdir
drwxr-xr-x  3 bcr  staff   96 13 Okt 12:37 subdir
$ ls -l newdir
ls -l newdir
total 16
-rw-r--r--  1 bcr  staff  4 13 Okt 12:01 baz
-rw-r--r--  1 bcr  staff  4 13 Okt 13:50 foo
```

Manipulating files and directories - Remove

A file is deleted from the filesystem using the `rm` (remove) command. Removing an empty directory is accomplished by `rmdir` (remove directory). A non-empty directory is removed with the recursive `-r` parameter of `rm`. It will delete all files and any subdirectories as well as the directory itself that is provided as target. The syntax is:

```
rm target
```

The `rm` command allows multiple target files or directories as parameters.

Be careful!

Whenever a file or directory is deleted, it is gone! There is no default recycle bin from where they can be restored on accidental deletion. Be careful when using `rm`, especially with the `-r` parameter. When in doubt, use `-i` to let `rm` ask you each time whether it should delete the file or directory in question.

Manipulating files and directories - Remove

Given our directory `subdir` with `foo` in it, delete the file with: `rm subdir/foo`

```
$ ls -l subdir
total 8
-rw-r--r--  1 bcr  staff  4 13 Okt 12:37 foo
$ rm subdir/foo
$ ls -l subdir
total 0
```

Now that the directory `subdir` is empty, delete it as well: `rmdir subdir`
With the recursive parameter of `rm`, both operations are combined into one:
`rm -r subdir/foo`

Absolute vs. Relative Addressing in the Filesystem

When dealing with filesystem-related commands such as `cp`, `mv`, `rm` and others, absolute or relative adressing are available.

Absolute Addressing

Always starts from the root (`/`) of the filesystem tree (absolute position).

Example: copy foo to /tmp

```
cp /home/bcr/foo /tmp
```

Relative Addressing

Starts from the current directory (relative position)

Example: Copy foo to subdir

```
cp foo subdir
```

Note:

Both absolute and relative addressing may be combined in a command:
`cp foo /tmp`
To address the current directory, use `.` (dot). For the directory above the current one, use `..` (two dots). Each subdirectory in a filesystem path is delimited with a `/` (slash).

Example: Move `myfile` from the current directory to the one above it

Overview

1 Introduction
Unix History
The Unix Philosophy

2 Overview
Logging into the Unix System
The Shell
Files in Unix
Getting help from the System

Getting help for commands

A Unix system contains commands and associated parameters in the base system alone. Even more programs from third parties are available. Making it nearly impossible to know all commands and parameter combinations, the system offers help in form of manual pages (man pages). For most commands there exists a man page which describes the command can do. Call up man pages using `man command`.

Order of man pages

Of course, the manual system itself has a man page, too: `man man`. The number in parentheses after a command describes the order as follows:

1. User commands
2. System calls and error numbers
3. Functions of the C library
4. Device drivers
5. File formats
6. Games and other entertainment
7. Miscellaneous information
8. System administration and -commands
9. Information for kernel developers

Example:
ping(8) is the man page of the `ping` command, i.e. `man 8 ping`. Some commands (i.e. `chmod`) appear in more than one man page (for example as a system call or as a user command). That's why an exact ordering is important.

Exercise

Use the command `mkdir` to create a directory called `d1`. Using cd(1), change into this directory. Determine the command usage from their respective manual pages.

1. Change into the directory `d1` you created
2. Use the cat(1) command to create a file `dat1`, which consists of lines of text you want.
3. Rename the file `dat1` to `dat2`.
4. Consult the man page for `cd(1)` and `cat(1)`.
5. How many man pages does the `mount` program have?

Further Information



[TDAIOTFOS] Marshall Kirk McKusick, George V. Neville-Neill
The Design and Implementation of the FreeBSD Operating System
Addison-Wesley © Pearson Education Inc.



Robert Watson
How a large scale opensource project works, FOSDEM 2008
`http://mirrors.dotsrc.org/fosdem/2008/maintracks/FOSDEM2008-largescale.ogg`



The FreeBSD Documentation Project
The FreeBSD Handbook, Basics chapter
`http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/basics.html`



Youtube
Stephen Bourne: Early days of Unix and design of sh
`https://www.youtube.com/watch?v=2kEJoWfobpA`